Mixture-of-Recursions: Learning Dynamic Recursive Depths for Adaptive Token-Level Computation

From Standard Transformers to MoR

Standard Transformer Overview

- Uses **L stacked layers** (each = self-attention + feed-forward).
- Update rule:
- $$h_t^{(l+1)} = f\left(h_t^{(l)}; \Phi^{(l)}\right)$$
- Where l∈ 0,1…L-1 (no.of layers),t is token number.
- Each layer has its own parameters Φ^l.

Drawbacks:

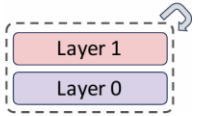
- Parameter-heavy: Millions to billions of weights (each layer has its own parameters).
- High memory footprint : Storing parameters + activations makes training and deployment expensive.
- High compute cost (FLOPs) :Self-attention scales quadratically with sequence length $O(N^2)$.
- Large KV cache requirement : During autoregressive decoding, all past tokens' keys and values must be cached for every layer, which becomes a major bottleneck for long sequences.

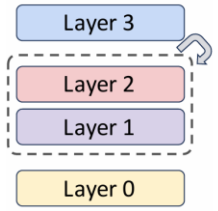
Recursive Transformers

- idea: Instead of using Lunique layers, reuse the same block of layers multiple times (recursion).
- Equation:

$$h_t^{(r+1)} = f\left(h_t^{(r)}; \Phi'\right)$$

- Where r ∈ 1,… Nr, Nr= recursion steps.
- Effective depth increases without increasing parameters.

	Cycle Strategy	
Layers	Equation	Figure
Last	–	
Recursion	$f\left(\mathbf{h}_t^{\ell}; \Phi'_{\ell \bmod (L/N_r)}\right)$	
First	–	

Middle-Cycle Strategy	
Equation	Figure
$f\left(\mathbf{h}_t^{L-1}; \Phi_{L-1}\right)$	
$f\left(\mathbf{h}_t^{\ell}; \Phi'_{(\ell-1 \bmod ((L-2)/N_r))+1}\right)$	
$f\left(\mathbf{h}_t^{0}; \Phi_0\right)$	

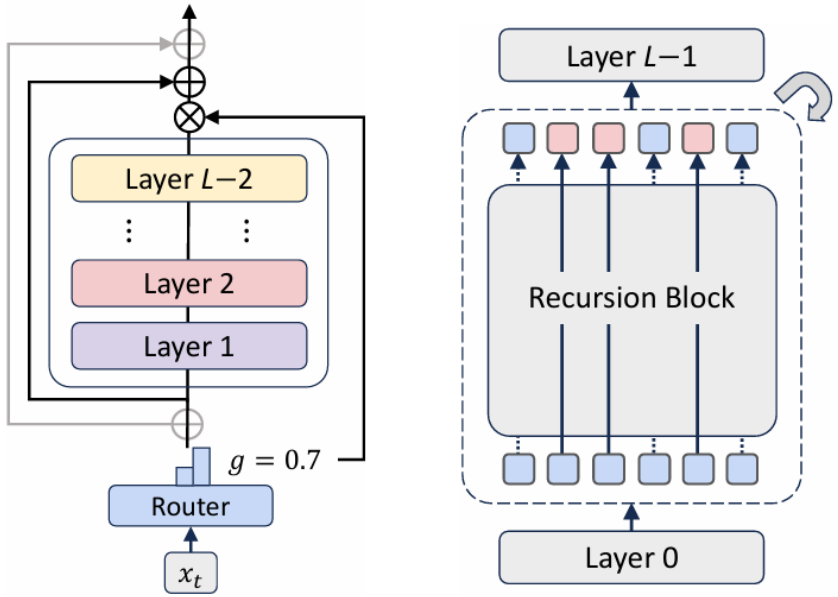
What it fixes from original Transformer:

- Reduces parameter count (weight sharing).
- Smaller memory footprint (fewer unique weights).

What it does not fix:

- Still applies same depth to every token (no adaptive computation).
- KV cache still grows large (since recursion creates multiple caches).
- Compute is still wasted on “easy” tokens that don’t need deep reasoning.

MoR (Mixture-of-Recursions)



Unifies two efficiency paradigms:

- Parameter sharing (recursive blocks).
- Adaptive computation (token-level routing).
- Uses routers to assign token-specific recursion depths.
- Equation:

$$H_t^{r+1} = g_t^r f(H_t^r; \Phi') + \text{(residual)}$$

- Adds KV caching for memory efficiency.

Two Routing Strategies:

- Expert-choice :Each recursion step picks its top-k tokens (step-wise decision).
- Token-choice : Each token commits to a recursion depth at the start (token-wise decision).

Two KV Caching Strategies:

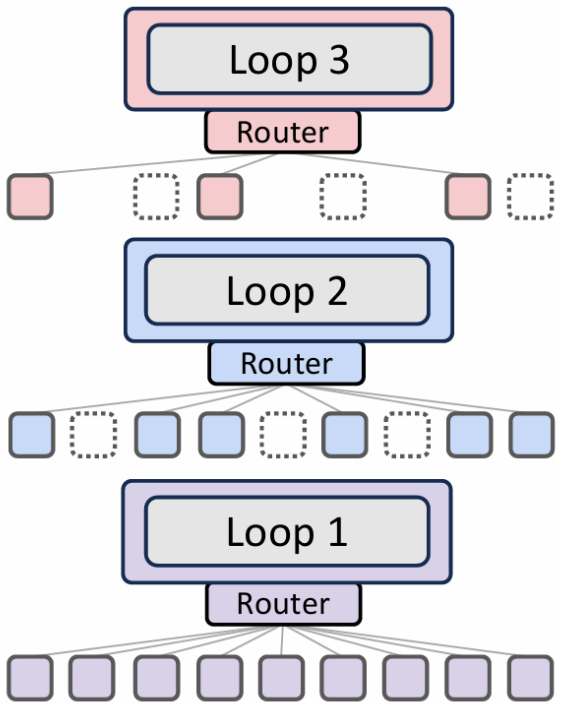
- Recursion-wise caching :Cache only active tokens at each recursion step.
- Recursive sharing : Cache all tokens once at the first step and reuse across recursions.

Expert-choice Routing

- Each recursion step = one “expert”.
- Router score:

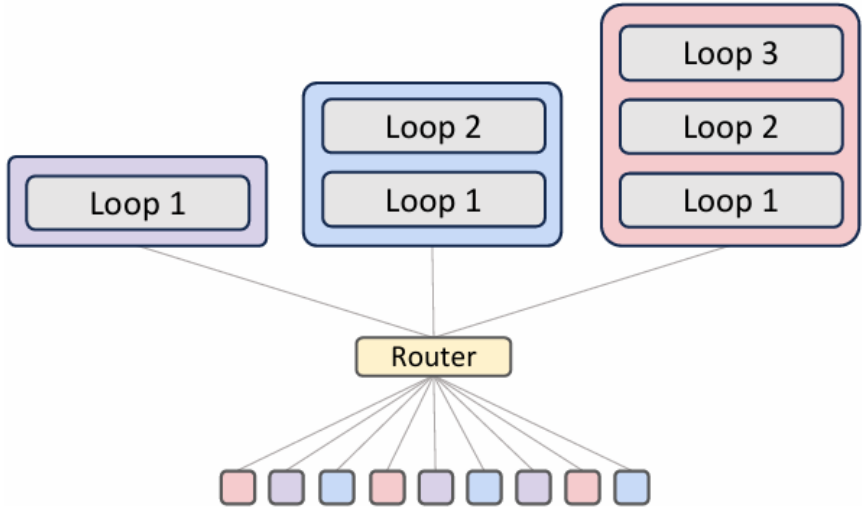
$$g_t^r = \mathcal{G}\left(\theta_r^{\top \mathcal{H}_t^r}\right)$$

- $\mathcal{G}$ represents an activation function.
- Top-k tokens are selected for next step.
- Residual: $$\mathcal{H}_t^{r+1} = \begin{cases} g_t^r f(\mathcal{H}_t^r, \Phi') + \mathcal{H}_t^r, & \text{if } g_t^r > P_\beta(G^r) \\ \mathcal{H}_t^r, & \text{otherwise} \end{cases}$$



- where $P_\beta(G^r)$ is the-percentile threshold over all scores at recursion step r

Token-choice Routing



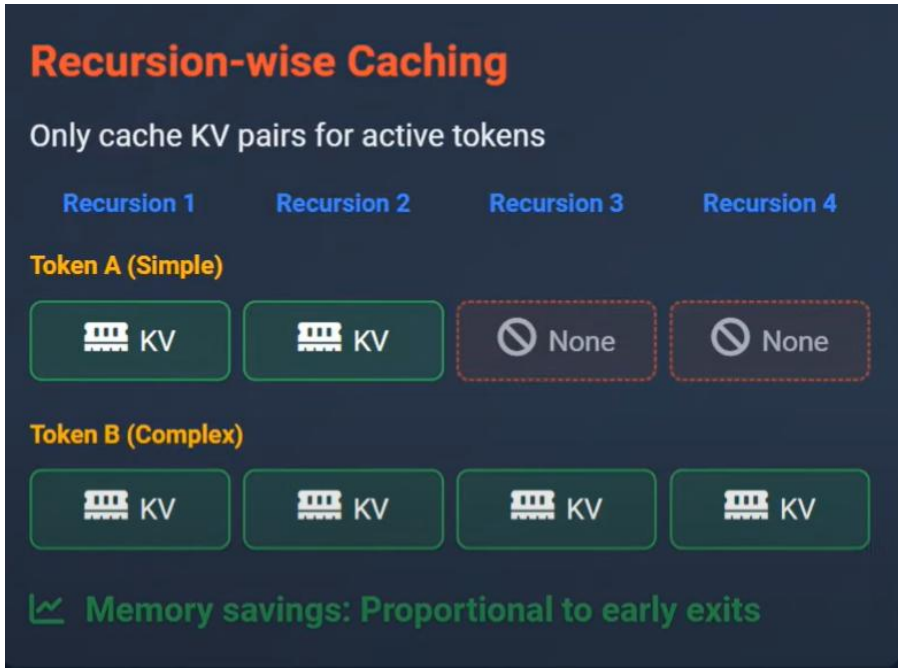
- Each token **commits once** to depth i.
- Decision: $i = \arg\max_j g_t^j$
- Residual: $$\mathcal{H}_t^{r+1} = \begin{cases} g_t^r f(\mathcal{H}_t^r, \Phi') + \mathcal{H}_t^1, & \text{if } r = i \\ g_t^r f(\mathcal{H}_t^r, \Phi'), & \text{otherwise} \end{cases}$$
- Here $\mathcal{G}$ is a non-linear function (softmax or sigmoid)
- $g_t = \mathcal{G}(\theta_r^\top \mathcal{H}_t^1)$ where gj t denotes the routing score for expert j ∈ {1,…. Nr}.

Pros and Cons

	Expert-choice	**Token-choice**
Pros	Static compute budget	No leakage
Cons	Causality violation	Load imbalance
⌞ Sol	Aux Rout, Aux Loss	Bal Loss, Loss-free

Recursion-wise Caching

•At each recursion step, only active tokens cache their key–value (KV) pairs.
•Tokens that exit early are not cached further.
•Attention is restricted to those cached tokens at that depth.
•Memory/IO cost ≈ $\frac{N_r+1}{2N_r}$.
•FLOPs ≈ $(kN_{ctx})^2$.
•$Where$ Nctx = context length
•(no.of tokens in the input sequence).



Pros:
•Substantially reduces FLOPs.
•Memory- and compute-efficient.

Cons:
•Tokens that exit early leave behind missing KVs, so later tokens can't attend to them.
•May lose useful context.

Recursive KV Sharing

- Cache all tokens once at the first recursion step.
- Reuse this cache across all deeper recursions.
- Even if tokens exit early their original KVs remain accessible.

Memory ≈ $1/N_r$

- FLOPs ≈ $k \,.\, N_{ctx}$

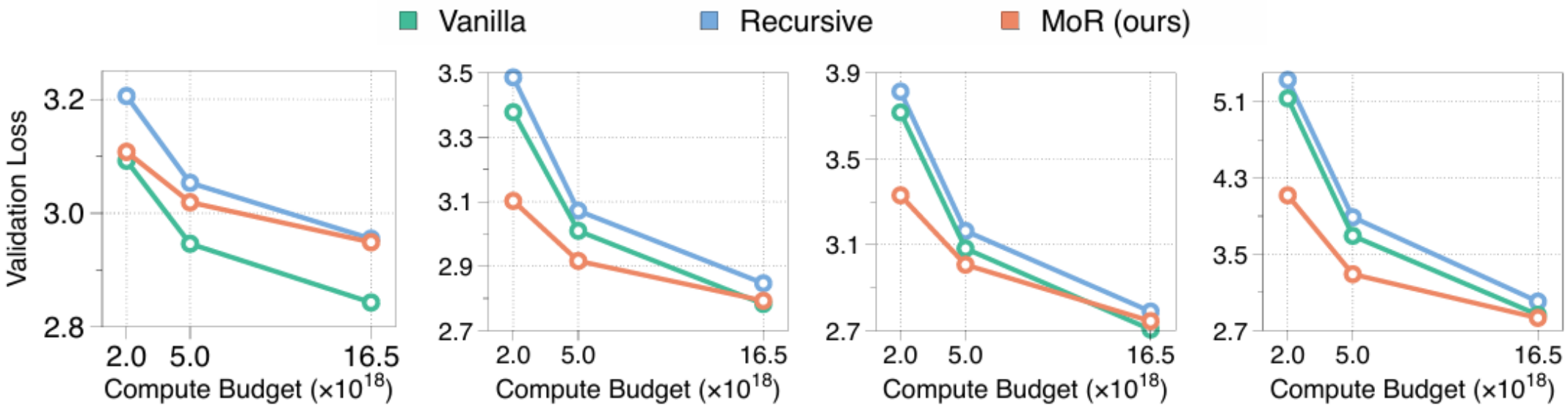
Pros:

- Preserves global context across all recursion steps.
- Avoids missing KVs (every token always available).
- No recomputation needed for dropped tokens.

Cons:

- KV I/O volume remains high, creating a bottleneck in decoding.
- Less FLOP reduction than recursion-wise caching.

	Recursion-wise Caching	**Recursive Sharing**
KV Memory	$(N_r + 1)/2N_r$	$1/N_r$
KV Cache IO	$(N_r + 1)/2N_r$	1
Attn FLOPs	k^2/N_{ctx}^2	k/N_{ctx}



(a) 135M-based model (b) 360M-based model (c) 730M-based model (d) 1.7B-based model

Validation loss across different compute budgets across four model sizes: 135M, 360M, 730M, and 1.7B parameters. For MoR models, we use expert-choice routing and recursion-wise caching. MoR consistently outperforms recursive baselines and matches or exceeds the standard Transformers at larger scales, despite using significantly fewer parameters (approximately one-third due to layer tying with NR = 3).